

Automated Kubernetes Workload Hardening Using a Functionality Oracle

"We just want to deploy securely — but end up fixing someone else's YAML"

Mathias Petermann <mathias.petermann@gmail.com>

Sebastian Graf <sebastian.graf@fhnw.ch>

School of Computer Science, FHNW

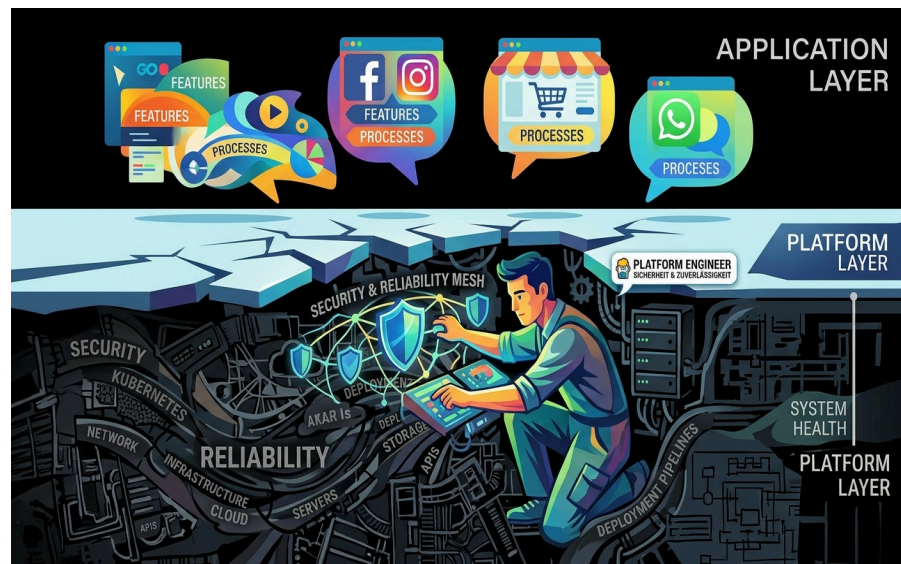
September 2026

Two Roles, one goal



"hey claude, generate me an application and push it to prod"

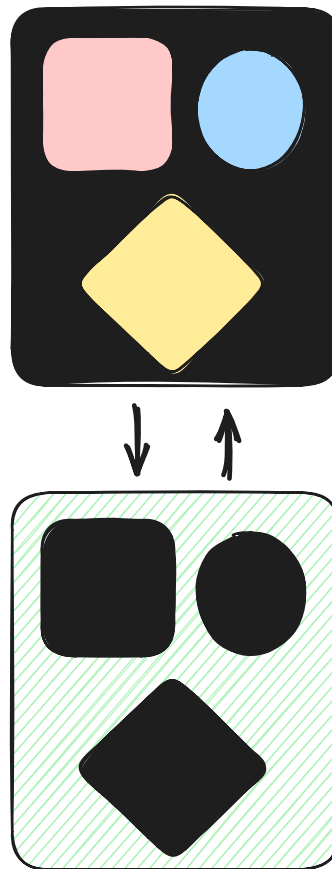
"I know that business value is created above... however I really would like to offer throughout reliable services."

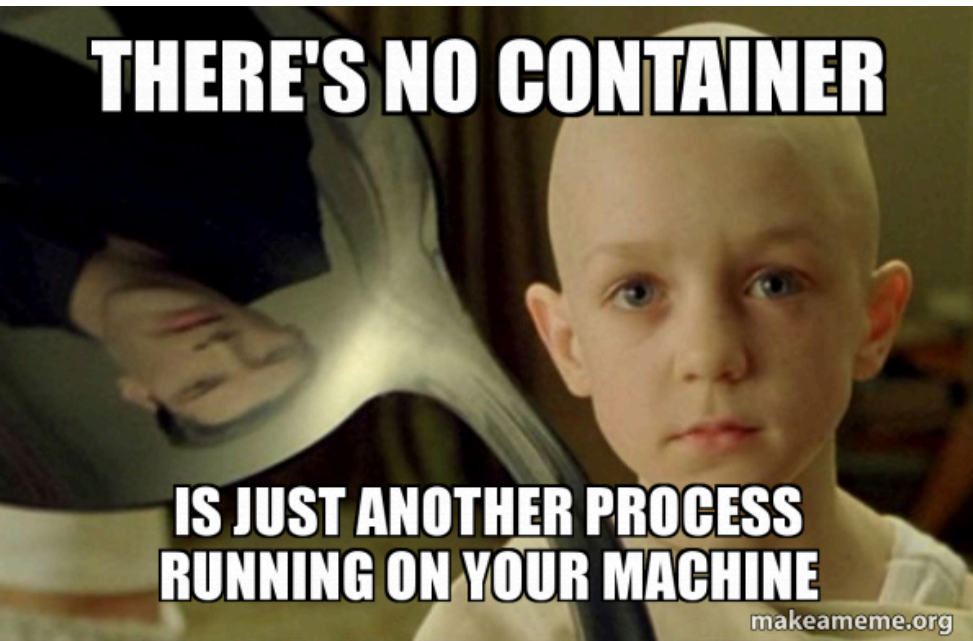


The gap

Responsibilities of Platform and Application is distributed over teams:

- ▶ on top: business teams focussing on application
 - ▶ focus on business requirements best
 - ▶ focus not on platform best practices
- ▶ on bottom: platform team, caring about platform for multiple applications
 - ▶ no / few application knowledge
 - ▶ need to guarantee integrity and resilience of entire platform





In Linux, how the kernel guardrails containers

Containers are not intended to isolate against the host without dedicated settings:

- ▶ on kernel-level:

`cgroups` , `chroot` , `namespaces`

If not set, workloads on the same host might interfere with each other.

In the K8s-world...

The fear of all platform engineering:

Multiple containers in various namespaces run on different nodes operated by distinct teams...

`securityContext` to the rescue:

Restrict the pod against the host is possible via security contexts (and mostly Linux capabilities under the hood).

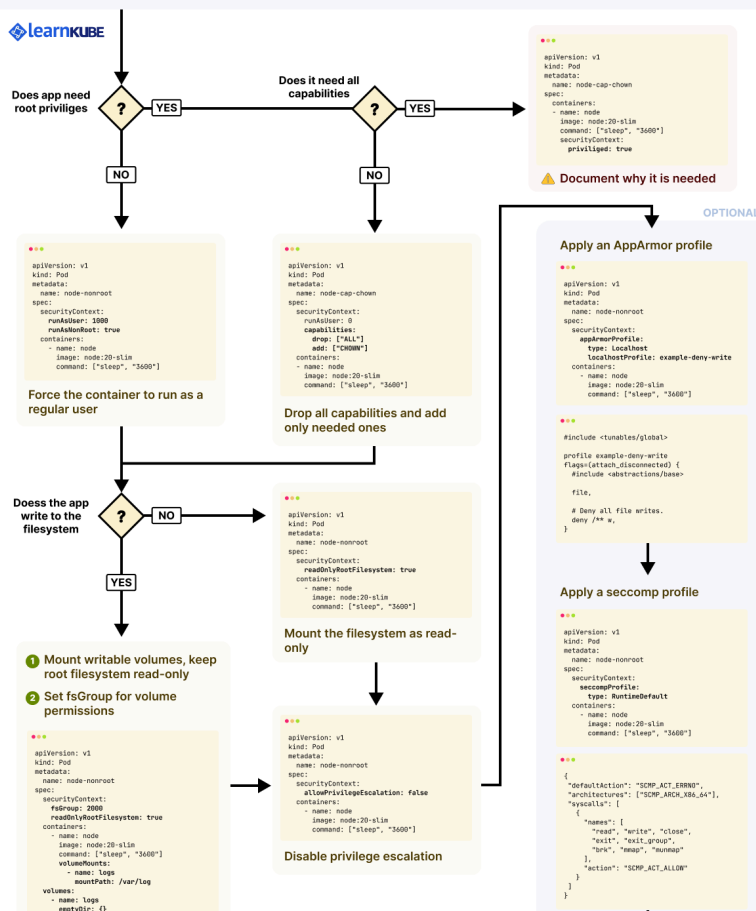
Field	Description
<code>allowPrivilegeEscalation</code>	Controls whether a process can gain more privileges than its parent, such as via <code>setuid</code> binaries. Container level only.
<code>capabilities</code>	Allows fine-grained control over Linux capabilities (e.g., <code>NET_ADMIN</code> , <code>SYS_TIME</code>). Container level only.
<code>fsGroup</code>	Defines a group ID used for setting group ownership of mounted volumes. Pod-level only.
<code>fsGroupChangePolicy</code>	Defines behavior of changing ownership and permission of the volume before being exposed inside Pod. Pod-level only.
<code>privileged</code>	Grants the container full access to the host, disabling most isolation mechanisms. Only available at the Container level.
<code>readOnlyRootFilesystem</code>	Mounts the containers root file system as read-only. Prevents write access to root-level paths. Container level only.
<code>runAsGroup</code>	Specifies the GID to run the container process. Useful for filesystem permissions. Applicable at Pod or Container level.
<code>runAsNonRoot</code>	Ensures the container does not run as root (user ID 0). Kubernetes will reject the Pod if no user ID is set. Applicable at Pod or Container level.
<code>runAsUser</code>	Specifies the UID to run the entrypoint of the container process. Applicable at Pod or Container level.

SECURING CONTAINERS IN KUBERNETES DEPLOYMENTS

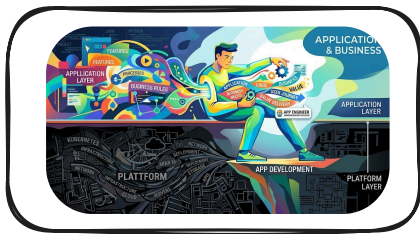
Choosing the correct
security context is hard ...

...even if you know the application well ...

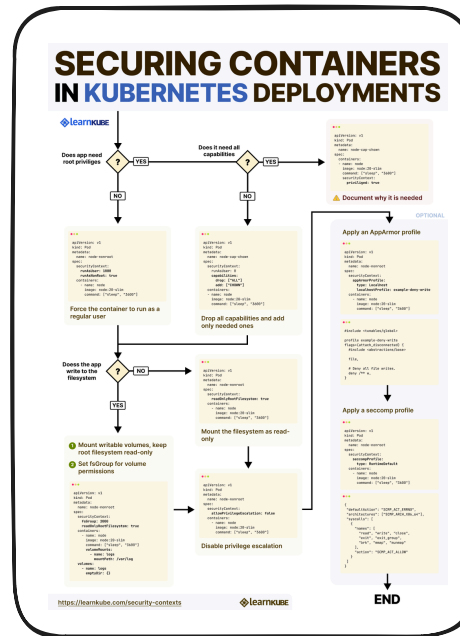
Ref: <https://learnkube.com/security-contexts>



To boil it down...



??/?



The idea

Treat functional correctness as a **black box** and **oracle** about its correctness:

1. Record a **baseline** of a known-good workload.
 2. Apply **one** runtime restriction at a time.
 3. **Compare** observed behavior against the baseline.
 4. Aggregate successful restrictions → recommended `securityContext`.
- No internal knowledge of the app required.
 - however...correct behaviour is assumed, not proved...
 - well...was software correctness ever proved?

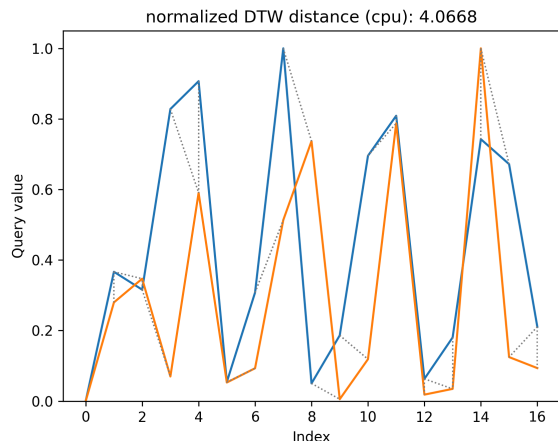


How "behaviour" of an app is defined

Category	Signals
Resource Metrics	▶ Collected per kubelet, 15s resolution. ▶ DTW evaluated → good for shifted patterns, but needs interpretation. ▶ Statistical summaries chosen: mean, median, std-dev, variance on normalized data. ▶ Metrics alone are not sufficient — used to spot outliers only.
Kubernetes Events	▶ Restarts, CrashLoopBackOff: only auxiliary context ▶ Probes alone prove "running", not "correct". Works as hard failure gate
Logs	▶ Container logs via kube-api ▶ Analyzing with Drain algorithm
App Metrics	▶ Theoretically, most relevant ▶ Endpoints are custom

Signal Resource Metrics - DTW

- ▶ Dynamic Time Wrapping (DTW) computes distance between points
- ▶ 15s resolution limits the reliability



Signal Resource Metrics - Statistics

- ▶ Statistic analysis over measured metrics
- ▶ Easier to compute / maintain than DTW
- ▶ Unfortunately similar to DTW not entirely reliable

	CPU			Memory		
Metric	Baseline	Test	Diff	Baseline	Test	Diff
mean	0.436	0.286	0.148	0.792	0.809	0.016
median	0.315	0.118	0.197	0.857	0.971	0.114
standard deviation	0.333	0.310	0.023	0.261	0.285	0.024
variance	0.111	0.096	0.014	0.068	0.081	0.013

Signal Logs - Drain Template Matching

- ▶ Parse logs into templates with the **Drain** algorithm.
- ▶ Train on **two** baseline recordings (so dynamic fields become `<*>`).
- ▶ Match check-run logs against baseline templates.
- ▶ Unmatched lines → anomalies; collapse recurring errors.
- ▶ Reliable even for workloads that stayed **Ready** — catches silent failures.

Baseline

```
time=2025-07-25T15:58:22.588Z level=INFO source=main.go:725 msg="Starting Prometheus Server"
mode=server version="(version=3.4.2, branch=HEAD,
revision=b392caf256d7ed36980992496c8a6274e5557d36)"

time=2025-07-25T15:58:22.592Z level=INFO source=main.go:1266 msg="Starting TSDB ..."

time=2025-07-25T15:58:22.817Z level=INFO source=head.go:862 msg="WAL replay completed"
component=tsdb checkpoint_replay_duration=42.820087ms wal_replay_duration=158.587917ms
wbl_replay_duration=138ns chunk_snapshot_load_duration=0s mmap_chunk_replay_duration=4.466189ms
total_replay_duration=205.908907ms
```

Test

```
time=2025-07-25T15:59:42.428Z level=INFO source=main.go:725 msg="Starting Prometheus Server"
mode=server version="(version=3.4.2, branch=HEAD,
revision=b392caf256d7ed36980992496c8a6274e5557d36)"

time=2025-07-25T15:59:42.432Z level=INFO source=main.go:1266 msg="Starting
TSDB ..."

time=2025-07-25T15:59:42.657Z level=INFO source=head.go:862 msg="WAL replay
completed" component=tsdb checkpoint_replay_duration=40.269496ms
wal_replay_duration=164.763458ms wbl_replay_duration=131ns
chunk_snapshot_load_duration=0s mmap_chunk_replay_duration=1.826402ms
total_replay_duration=206.89316ms
```

Drain Template

```
<*> level=INFO source=main.go:725 msg="Starting Prometheus Server" mode=server
version="(version=3.4.2, branch=HEAD, revision=b392caf256d7ed36980992496c8a6274e5557d36)"

<*> level=INFO source=main.go:1266 msg="Starting TSDB ..."

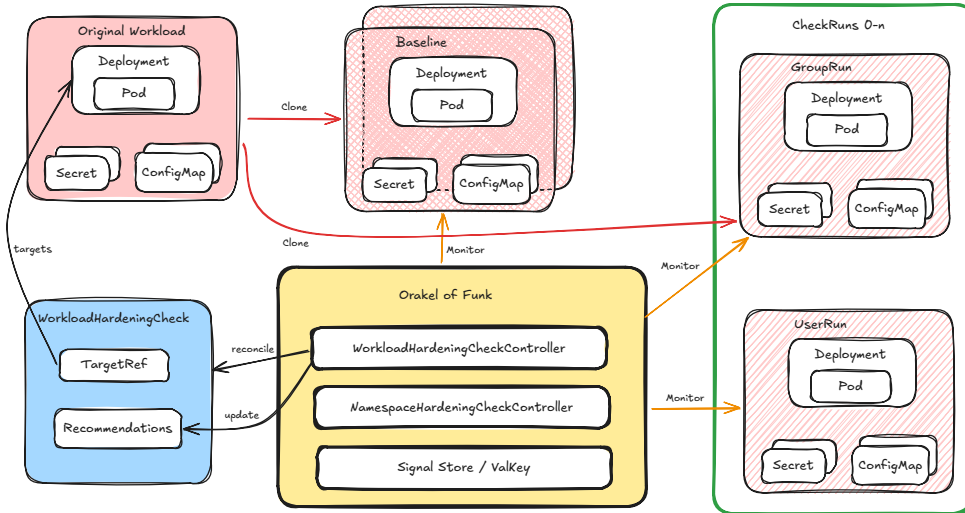
<*> level=INFO source=head.go:862 msg="WAL replay completed" component=tsdb <*> <*>
chunk_snapshot_load_duration=0s <*> <*>
```

Orakel Architecture

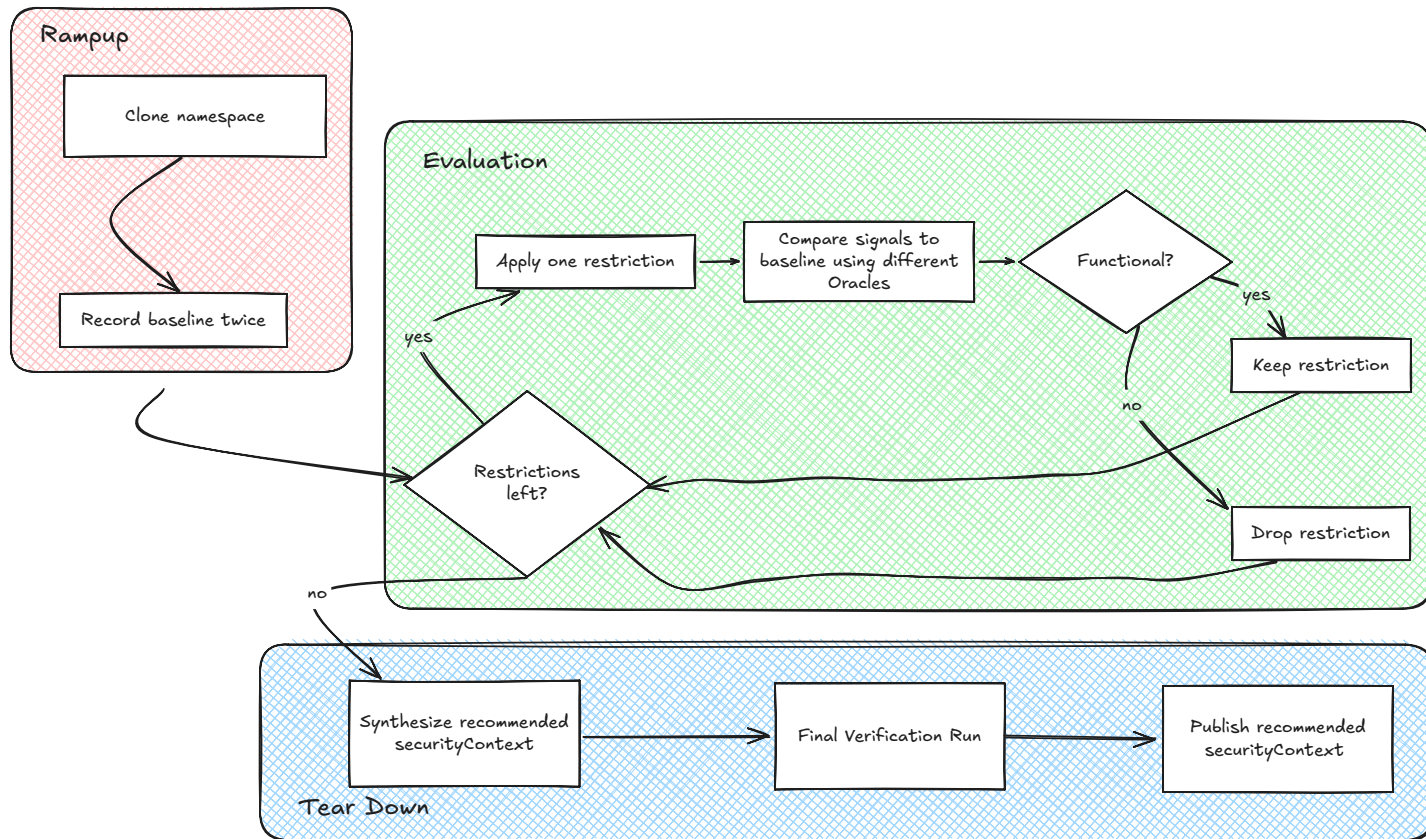
- ▶ **Operator SDK** Kubernetes operator, two CRDs:

- ▶ `WorkloadHardeningCheck` — one workload
- ▶ `NamespaceHardeningCheck` — all workloads in a namespace

- ▶ All runs execute in **cloned namespaces** (isolation preserved).
- ▶ Status tracked via `StatusConditions`
- ▶ Signals stored in **ValKey**



The Orakel Loop



Tests for Pod

ID	Title	Description	Datasource
TC-01	Pod Stability	Detect if the container terminated unexpectedly during execution.	Probes
TC-02	Pod Readiness	Identify if the container fails its healthiness probe during the test window.	Probes
TC-03	Log Pattern Matching	Compare runtime logs to baseline and detect anomalies.	Logs

- ▶ If any test-case fails $\Rightarrow$ current configuration is too restrictive

Workload Hardening Check

```
apiVersion: checks.funk.fhnw.ch/v1alpha1
kind: WorkloadHardeningCheck
metadata:
  name: nginx-unprivileged
  namespace: oof-nginx-unprivileged
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-unprivileged
  recordingDuration: 1m
```

- ▶ Check targets a specific workload
- ▶ The operator identifies the top-level owner objects for cloning.
 - ▶ This ensures that only controller-managed workloads (e.g., Deployments, StatefulSets) are evaluated, avoiding lower-level ReplicaSets or pods.

Namespace Hardening Check

```
apiVersion: checks.funk.fhnw.ch/v1alpha1
kind: NamespaceHardeningCheck
metadata:
  labels:
    app.kubernetes.io/name: podtato
  name: podtato
spec:
  targetNamespace: podtato-kubect1
  recordingDuration: 1m
```

- ▶ Workloads in the Namespace are automatically detected.
 - ▶ For each detected workload a Workload HardeningCheck is created.
- ▶ Currently only Deployment, StatefulSet and DaemonSet resources are supported

Result

```
conditions:
- lastTransitionTime: "2025-08-13T16:51:36Z"
  message: Finished, recommendation ready
  reason: Finished
  status: "True"
  type: Finished
recommendation:
  containerSecurityContexts:
    allowPrivilegeEscalation: false
    capabilities:
      drop:
      - ALL
    readOnlyRootFilesystem: true
    runAsGroup: 1000
    runAsUser: 1000
  podSecurityContext:
    fsGroup: 1000
    runAsGroup: 1000
    runAsNonRoot: true
    runAsUser: 1000
```

After all checks have passed, the operator synthesizes a recommended securityContext:

- ▶ podSecurityContext: for pod-level attributes
- ▶ securityContext: for container-level attributes

For validation, the namespaces with the workload is cloned one more time and the recommended securityContext is applied.

This FinalCheckRun must pass all test cases before the recommendation is published.

Can't we use LLM for that?

- ▶ LLMs are not deterministic
- ▶ Tests in early 2025 (GPT-4) showed a huge variance in results
- ▶ Not enough data to train a model for a single application
- ▶ Recent models are still not deterministic but produce more reliable results
- ▶ The concept itself can work with todays models
- ▶ Adding an LLM based Oracle could be done

Evaluation Workloads

- ▶ **Real-world:** Prometheus, ArgoCD, MariaDB, Podtato-Head (official Helm charts).
- ▶ **NGINX scenarios:** root, non-root unprivileged, read-only root + emptyDir.
- ▶ **Purpose-built:** chown, privilege escalation, filesystem writes, port binding.

Key Findings

- ▶ Stateless / loosely coupled workloads easiest to harden.
- ▶ Multi-component apps need namespace-level checks.
- ▶ **Log heuristics detected deviations even when pods stayed Ready.**
- ▶ Runtime variance matters (e.g. unprivileged port binding differs by runtime).
- ▶ Stateful workloads can be hardened with careful cloning.

Limitations

- ▶ Concurrency complicates status updates & resource versioning.
- ▶ Complex topologies (distributed systems, CRDs) hard to clone faithfully.
- ▶ Evaluation duration balances coverage vs. feedback loop.
- ▶ Metrics alone insufficient for functional correctness.

Takeaways

- ▶ Functionality-based hardening is **feasible** with minimal assumptions.
- ▶ Logs + probes + application metrics together form a robust oracle.
- ▶ Operator integrates natively, gives actionable workload-agnostic recommendations.

<https://github.com/orakel-of-funk>



Questions?

[Backup] Check Design

- ▶ **Isolated:** `readOnlyRootFilesystem`, `allowPrivilegeEscalation`, `capabilities.drop`
- ▶ **Grouped:** `runAsUser` / `runAsGroup` / `fsGroup` / `runAsNonRoot` set consistently
- ▶ Execution modes: **sequential** or **parallel**; `recordingDuration` configurable

[Backup] Future Work

- ▶ Dedicated **CLI** for CI/CD integration
- ▶ Separate `WorkloadHardeningReport` resource for cleaner reporting
- ▶ Consolidated baseline for namespace-level checks
- ▶ **LLM-based** semantic log analysis
- ▶ Regression testing across upgrades; broader CRD / distributed workload support